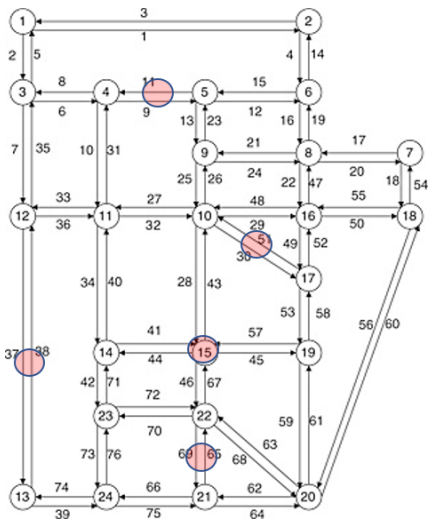
Bayesian Optimization Framework for Urban Transportation System Simulators

MUMS Transition Workshop at SAMSI

Laura Schultz and Vadim Sokolov

May 14, 2019

Transportation Simulator



- outputs: flows (passenger, transit, vehicles)
- x = parameters we are certain about (day of week, special event attributes, season)
- θ = unobserved parameters, e.g. traveler's preferences value of time, transit bias, ...

$$\boxed{\text{outputs} = \phi(x, \theta)}$$

But do they accurately depict the real world?

Objective and Challenges

Relationship between field observations y and the simulated values:

$$y_i = \phi\left(x_i, \theta\right) + \epsilon\left(x_i\right) + e_i$$

Use Maximum Likelihood Estimate we want to fit predicted flows $\phi(\theta)$ to average field traffic conditions $\hat{y}$

$$\text{minimize}_\theta \quad ||\hat{y} - \phi(\theta)||_2^2$$

Challenges

- No derivatives
- Simulator is stochastic
- Each simulator run takes few hours on a fast desktop
- θ is high dimensional

Bayesian Approach

$$L(\theta, y, x) = \quad \frac{1}{N}\sum_{i=1}^{N} ||y_i - \phi(x_i, \theta)||_2^2 \rightarrow \text{minimize}$$

1. Put prior on continuous functions $C[0, \infty]$
2. Repeat for $k = 1, 2, \ldots$
3. Evaluate ϕ at $\theta_1^k, \ldots, \theta_{n_k}^k$
4. Compute a posterior (integrate)
5. Decide on the next batch of points to be explored $\theta_1^{k+1}, \ldots, \theta_{n_{k+1}}^{k+1}$

Posterior minima is the solution to our optimization problem

Bayesian Optimization: Gaussian Process

Use Gaussian process surrogate for the loss function

$$L(\theta, x, y) \sim GP(m(\theta), k(\theta \mid x, y, \gamma) + \sigma^2 \delta_\theta)$$

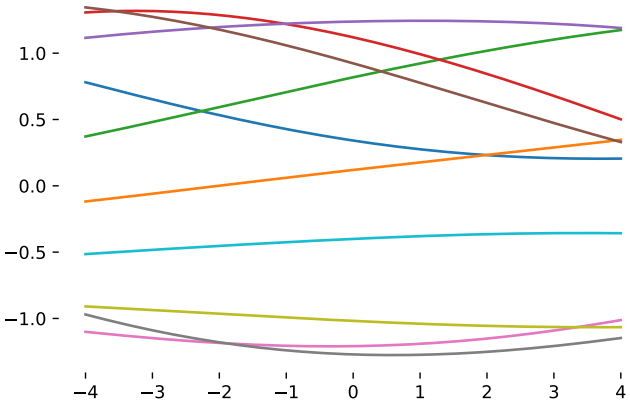
$$m(\theta) = E[L(\theta)], \qquad k(\theta, \theta') = E[(\theta - m(\theta))^T (\theta' - m(\theta')]$$

Typical choice exponentially decaying relation

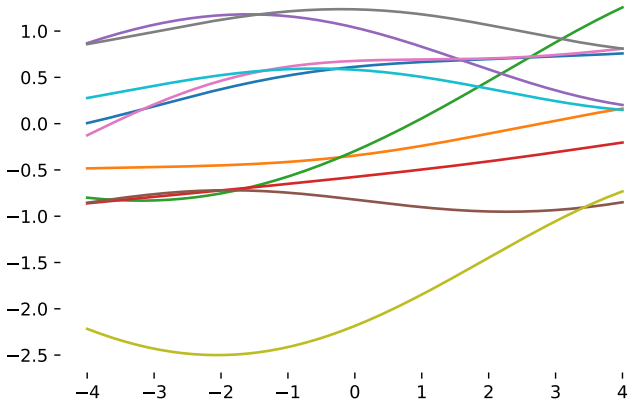
$$k_{SE}(\theta, \theta' \mid \gamma) = \sigma^2 \exp\left[-\frac{1}{2}\left(\frac{\theta - \theta'}{\lambda}\right)^2\right]$$

$\gamma = (\sigma^2, \lambda)$

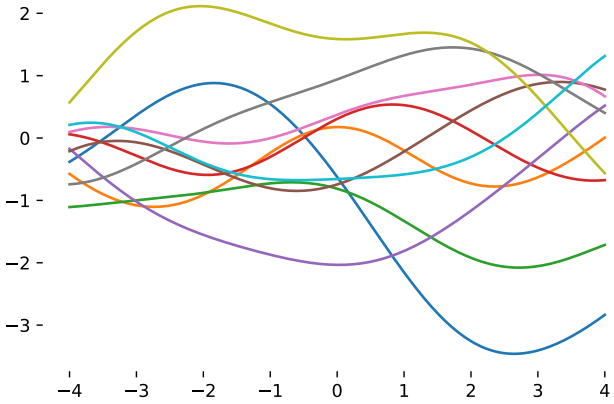
Gaussian Process: Prior



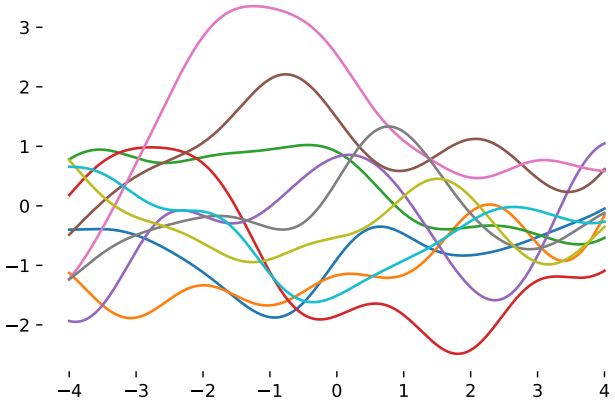
$\lambda = 10, \quad \sigma = 1$



$\lambda = 5, \quad \sigma = 1$

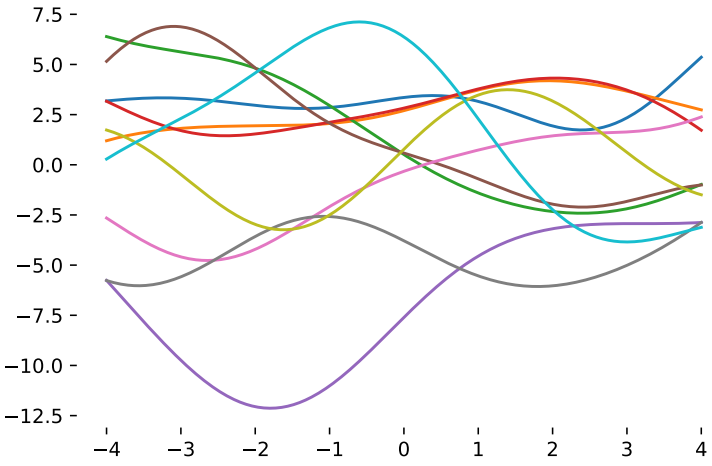


$\lambda = 2, \quad \sigma = 1$

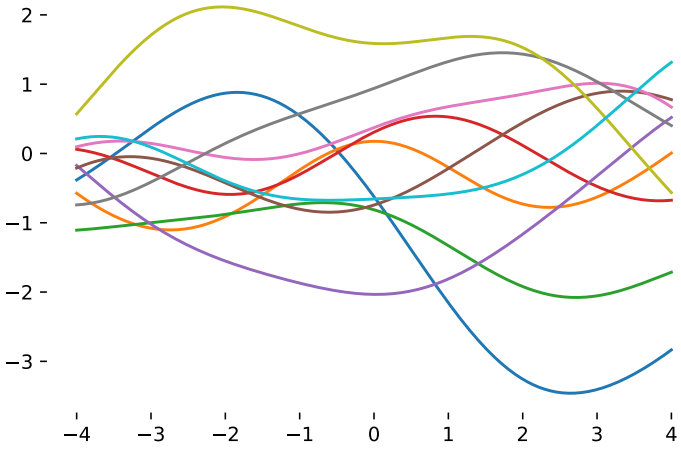


$\lambda = 1, \quad \sigma = 1$

Gaussian Process: Prior



$\lambda = 2, \quad \sigma = 16$



$\lambda = 2, \quad \sigma = 1$

Bayesian Optimization: Gaussian Process

Use Gaussian process surrogate for the loss function

$$L(\theta, x, y) \sim GP(m(\theta), k(\theta \mid x, y, \gamma) + \sigma^2 \delta_\theta)$$

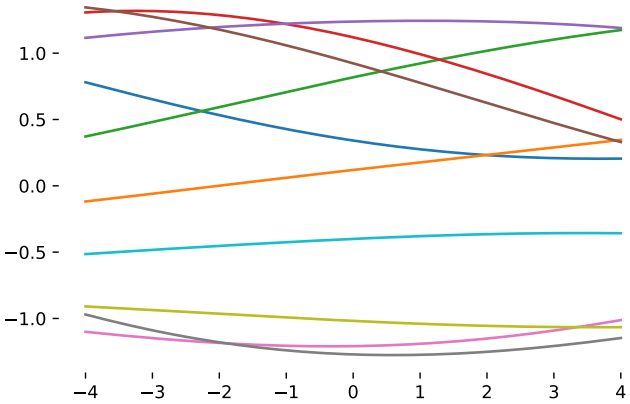
$$m(\theta) = E[L(\theta)], \qquad k(\theta, \theta') = E[(\theta - m(\theta))^T (\theta' - m(\theta')]$$

Typical choice exponentially decaying relation

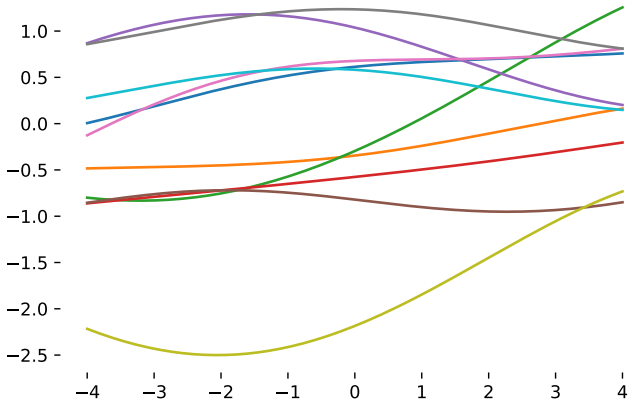
$$k_{SE}(\theta, \theta' \mid \gamma) = \sigma^2 \exp\left[-\frac{1}{2}\left(\frac{\theta - \theta'}{\lambda}\right)^2\right]$$

$\gamma = (\sigma^2, \lambda)$

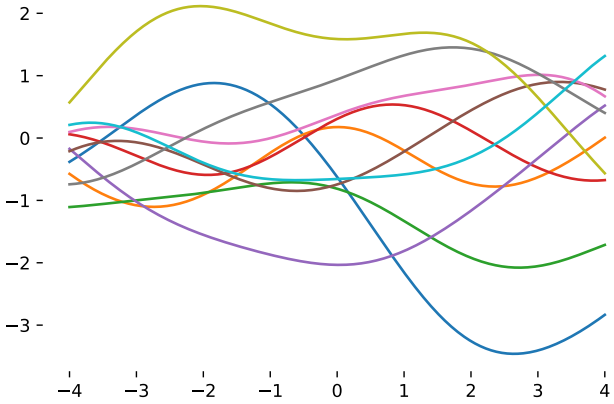
Gaussian Process: Prior



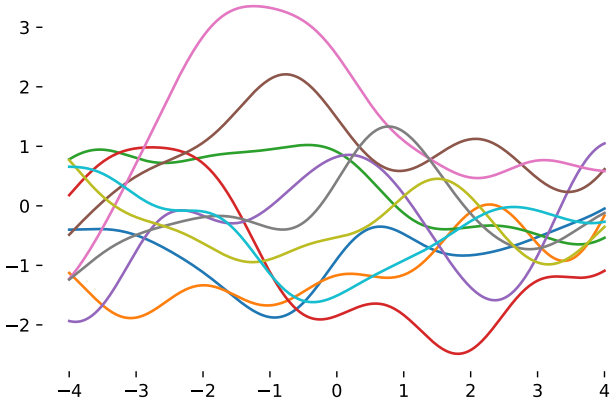
$\lambda = 10, \quad \sigma = 1$



$\lambda = 5, \quad \sigma = 1$

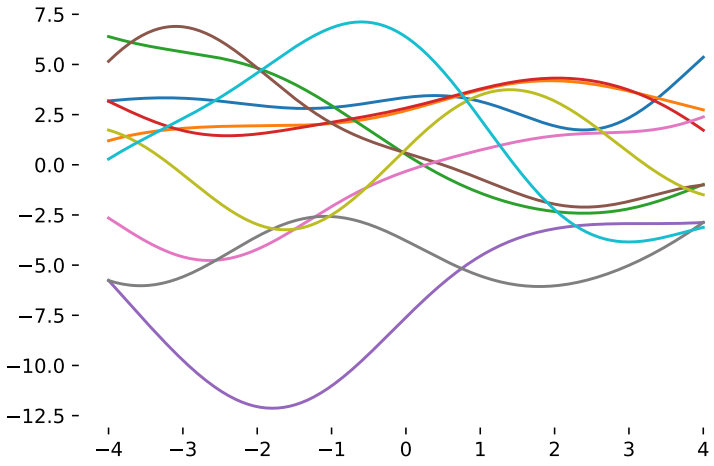


$\lambda = 2, \quad \sigma = 1$

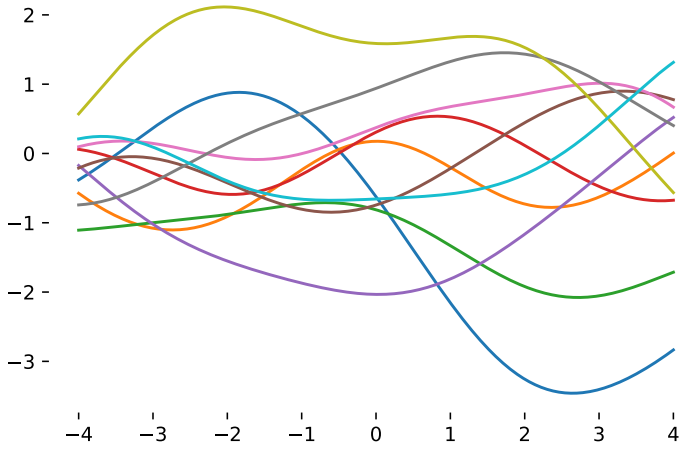


$\lambda = 1, \quad \sigma = 1$

Gaussian Process: Prior



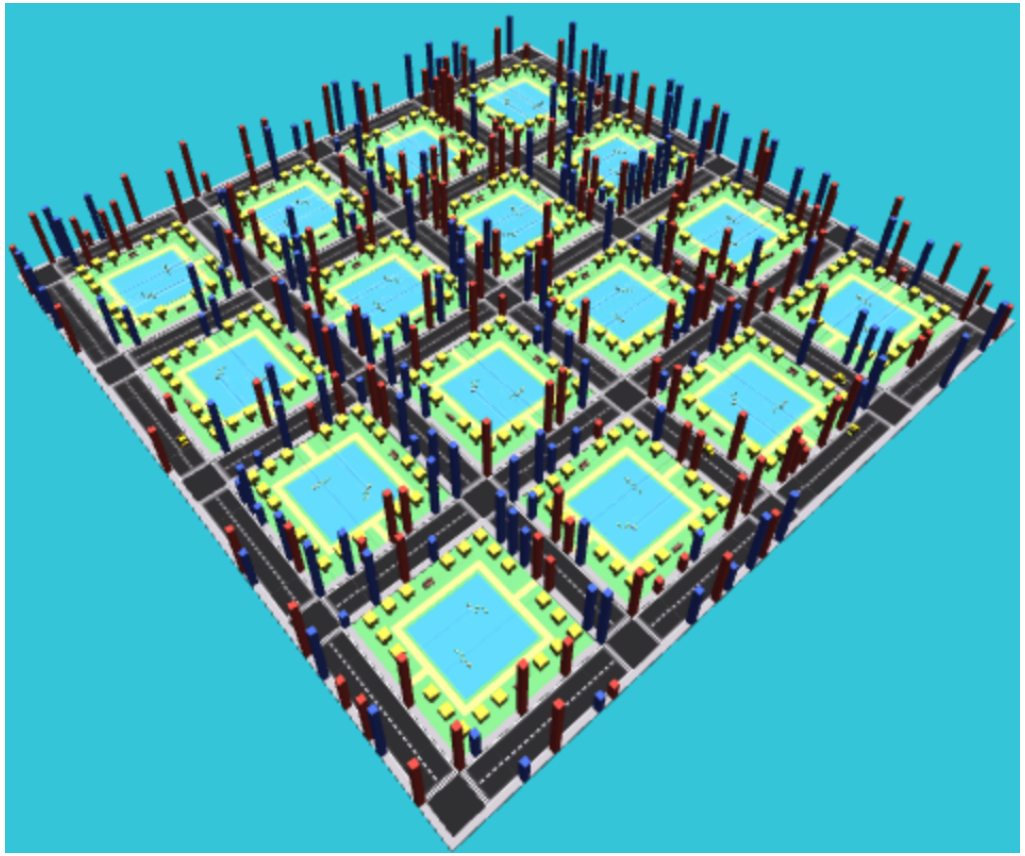
$\lambda = 2, \quad \sigma = 16$



$\lambda = 2, \quad \sigma = 1$

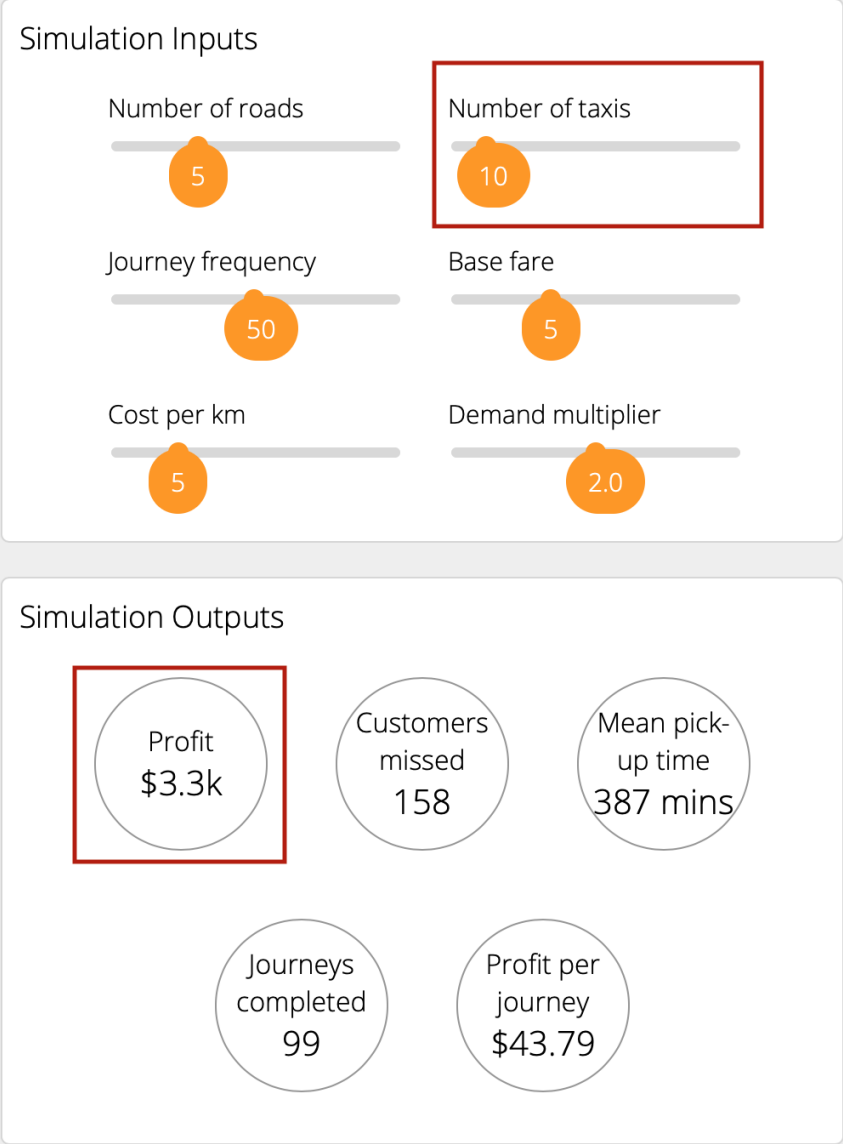
Taxi Example

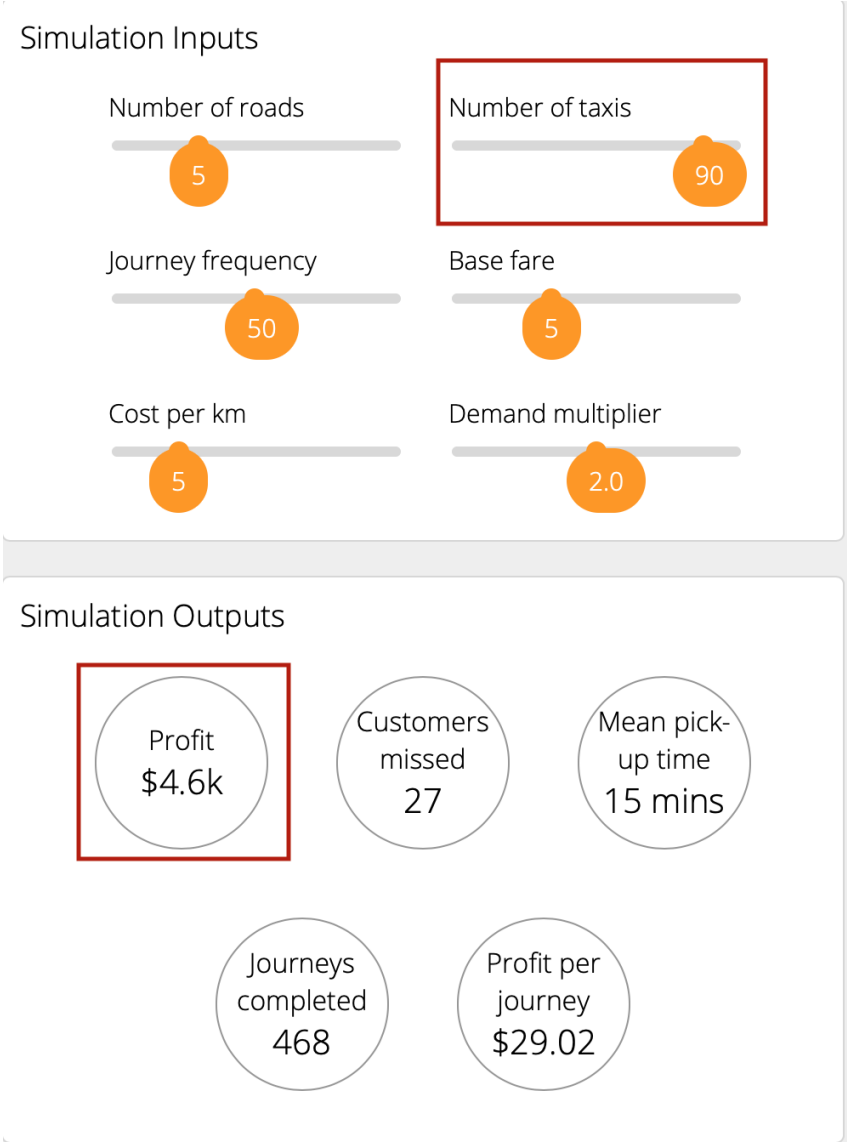
We use Emukit Playground. It simulates operations of a taxi fleet.



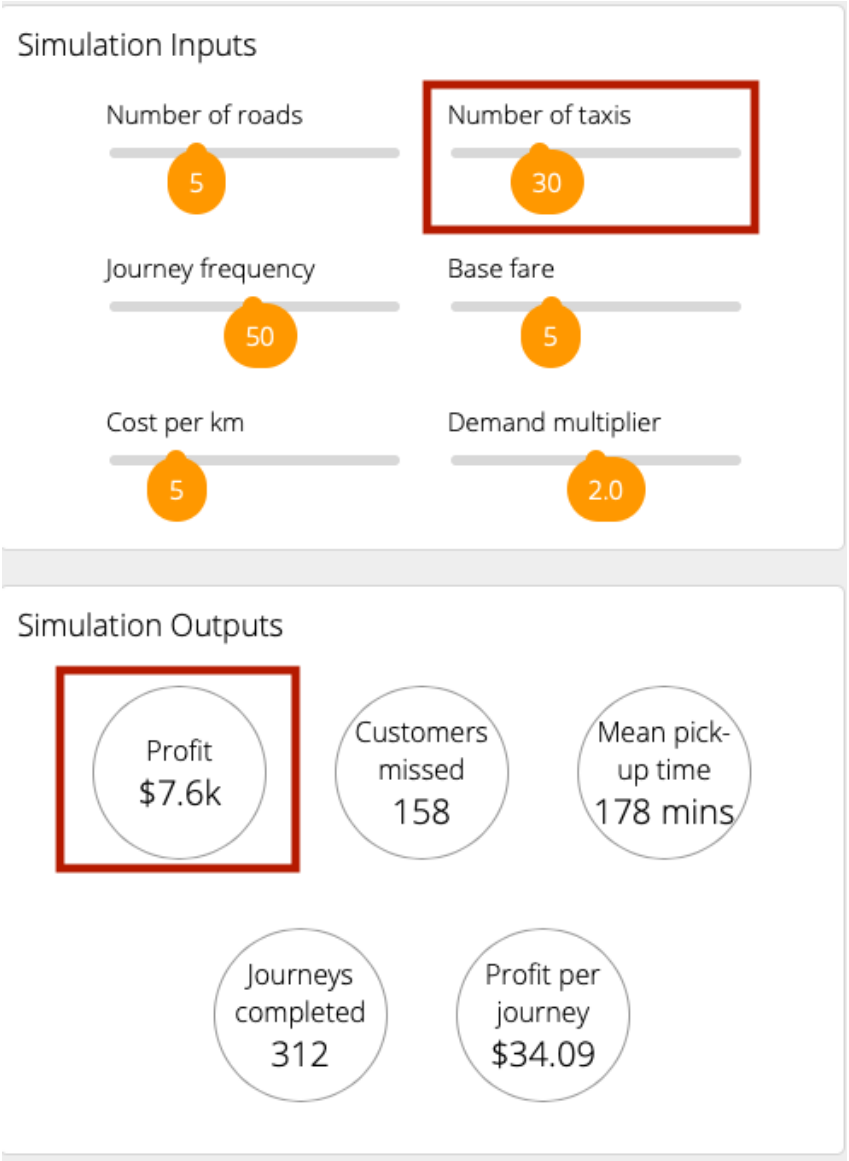
$$\text{Profit} = \phi(\text{Number of Taxis}) \rightarrow \text{maximize}$$

Taxi Example: Evaluate at Extremes

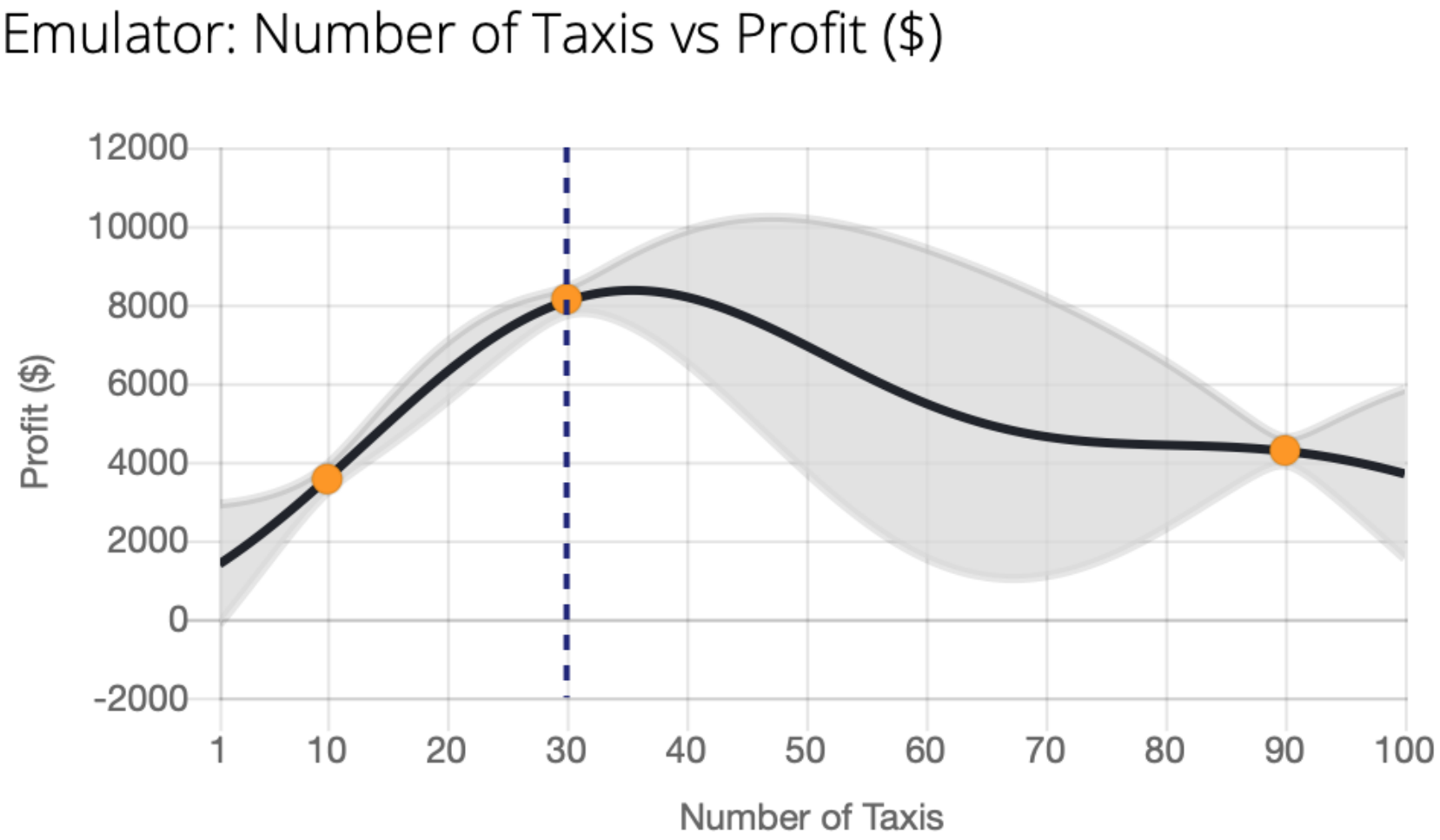




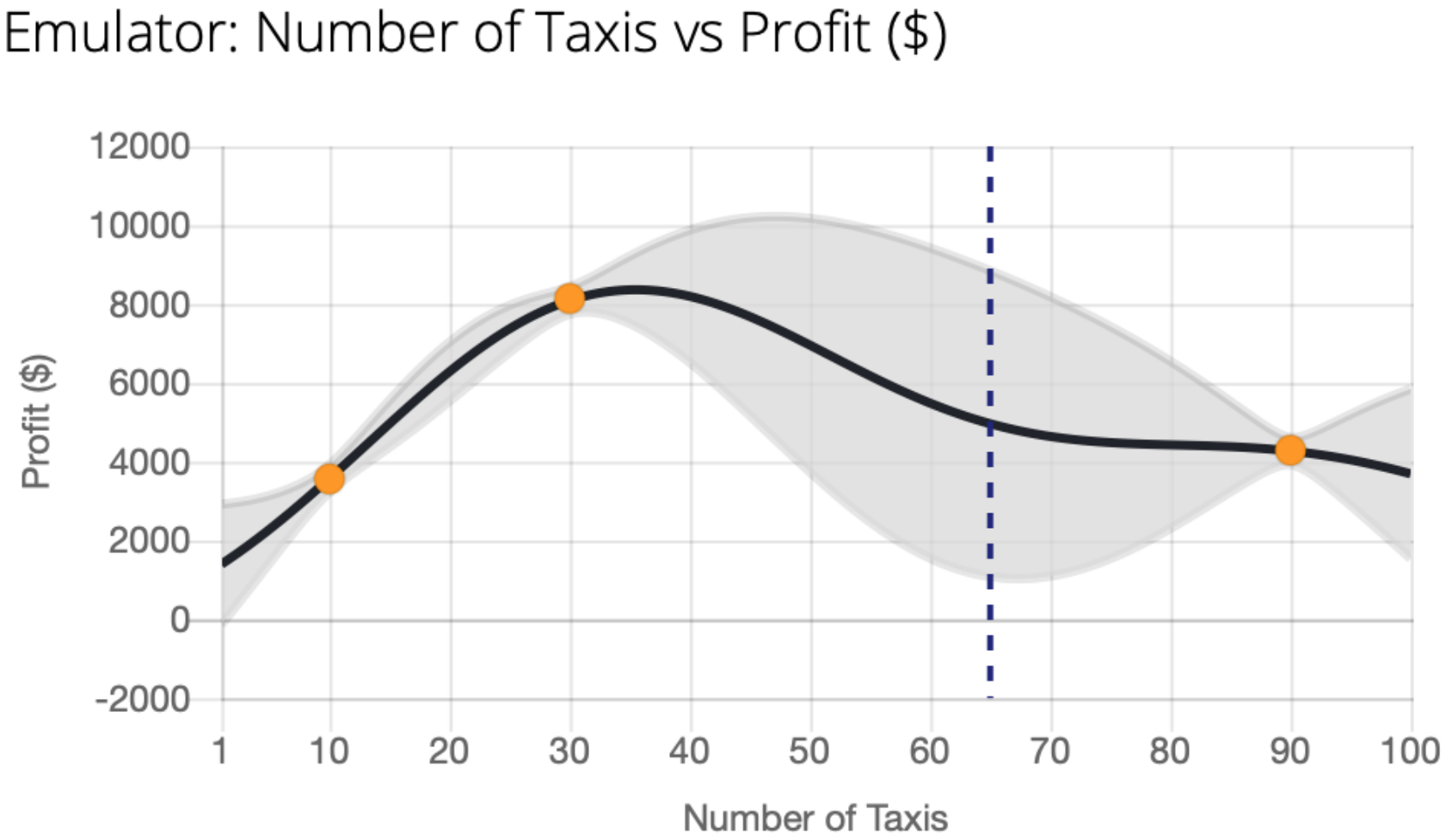
Taxi Example: Evaluate in the Middle



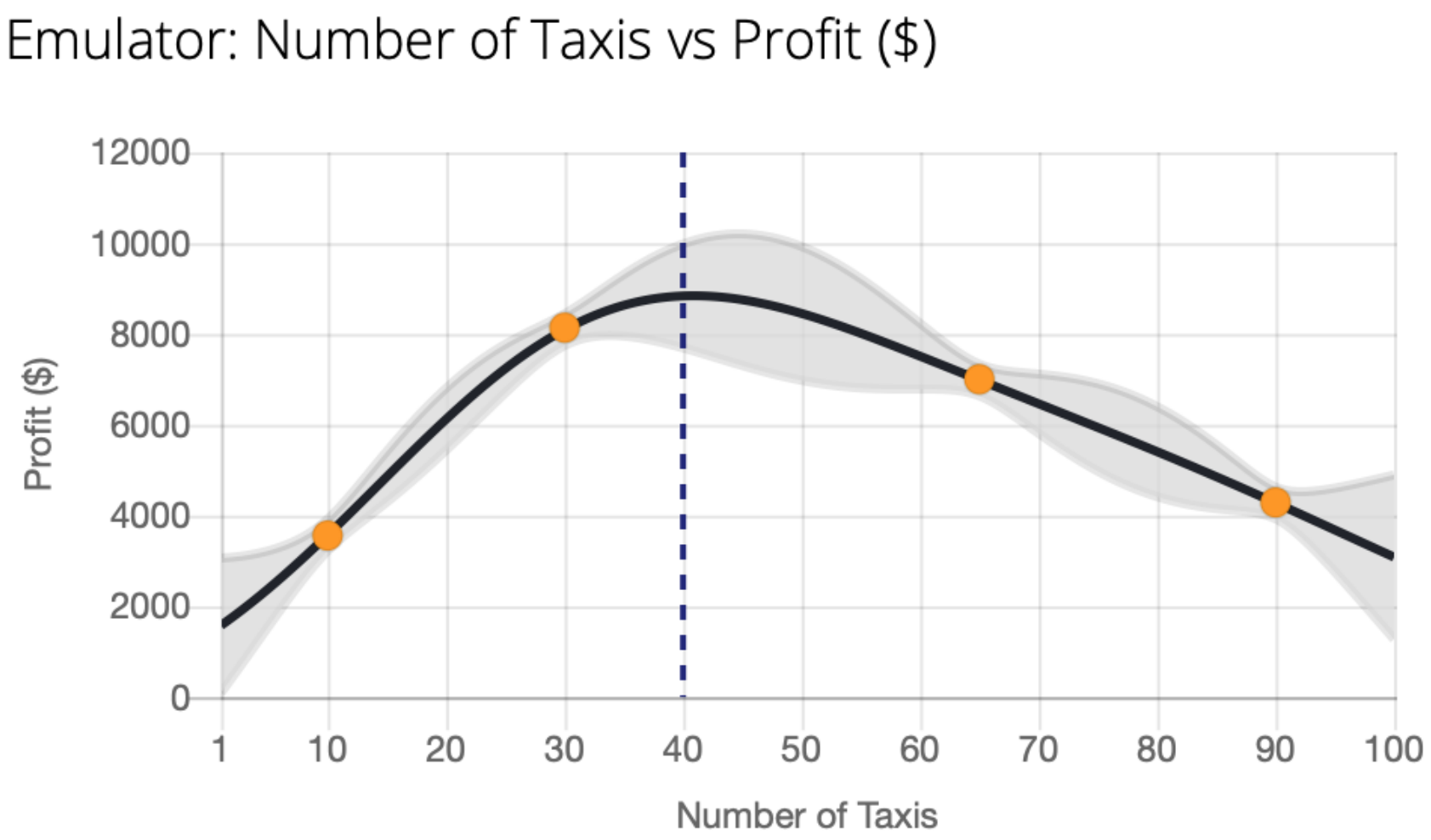
Taxi Example: Calculate the Posterior



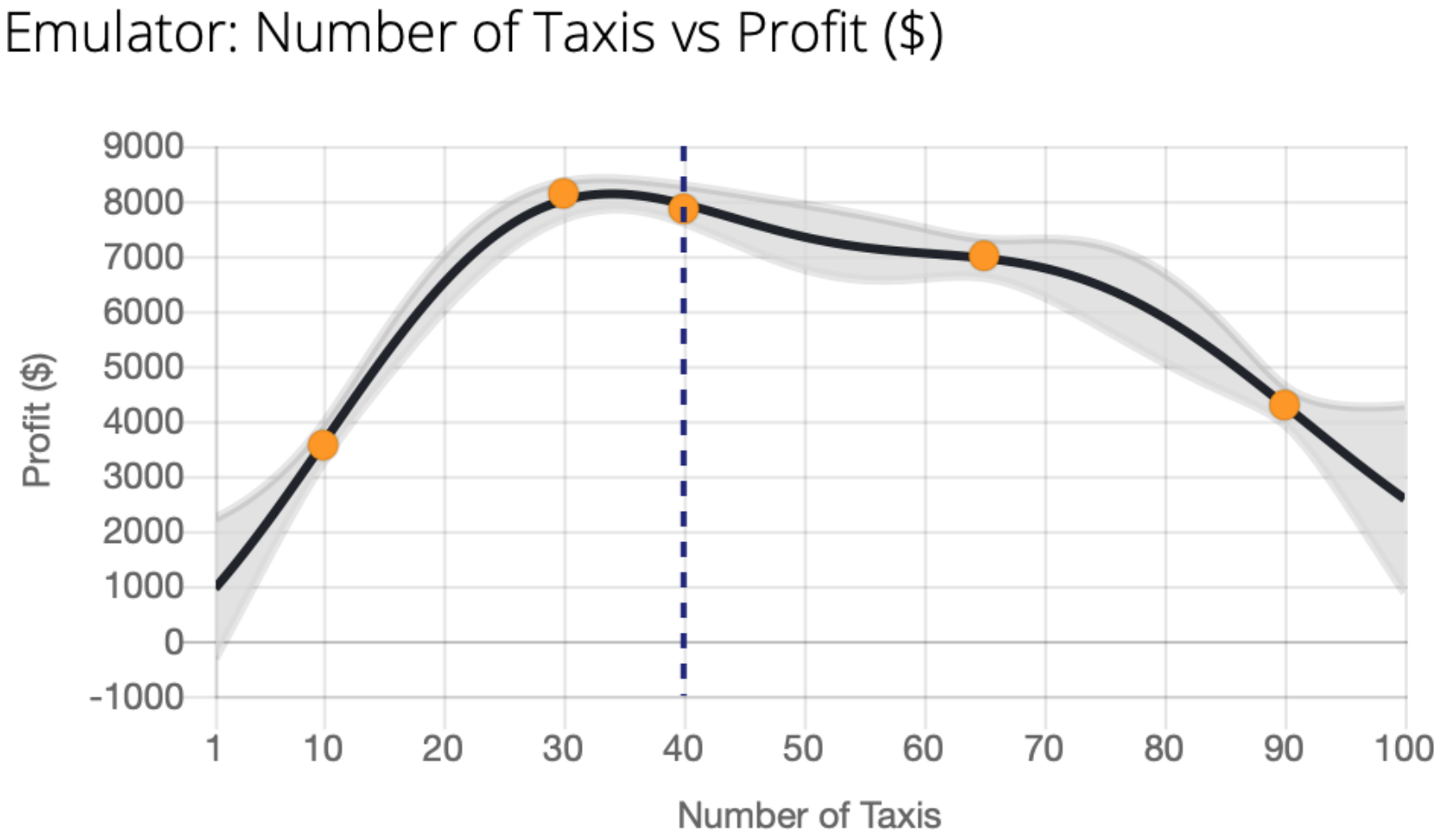
Taxi Example: Optimization Algorithm Takes Over



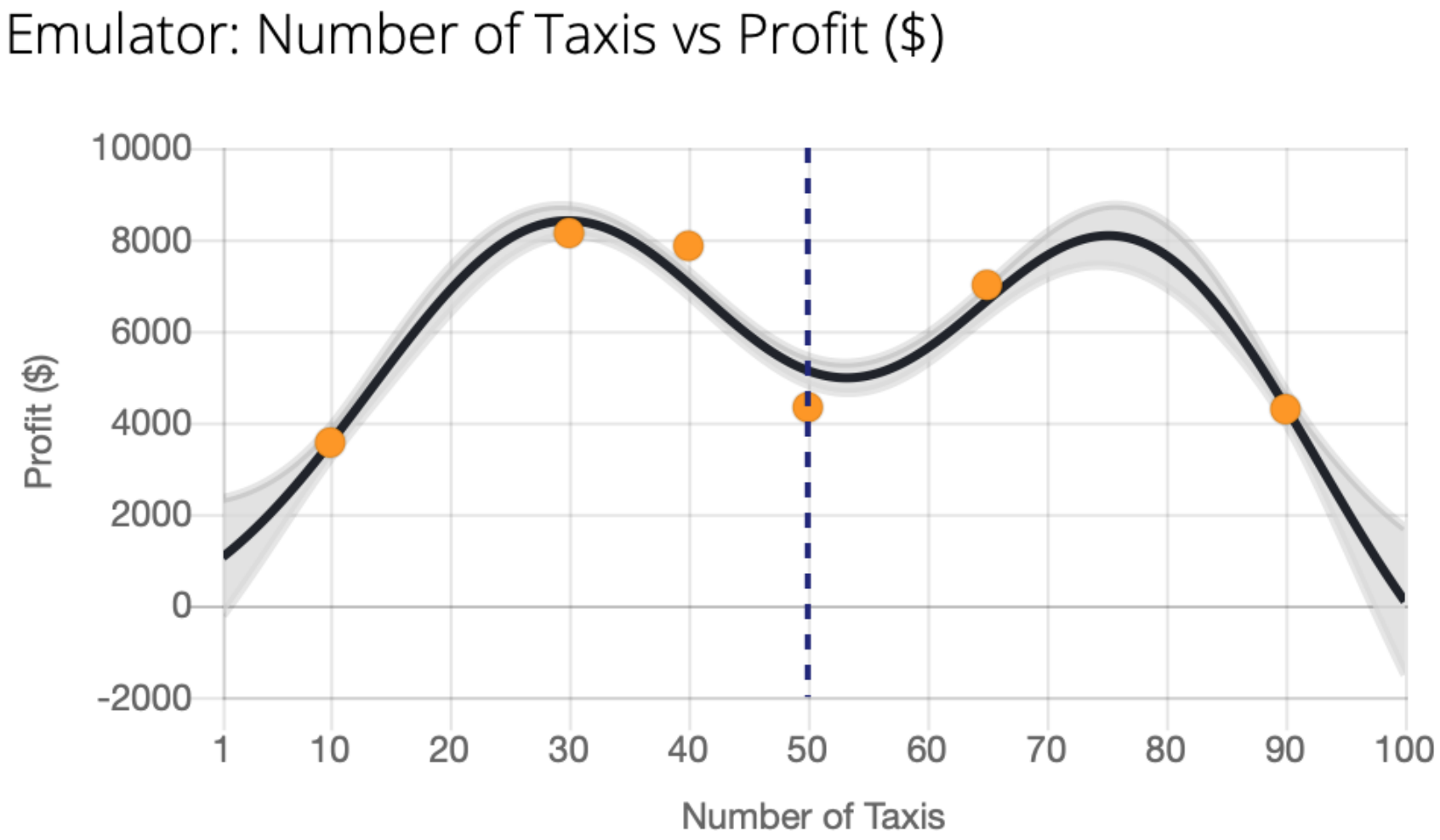
Taxi Example: Optimization Algorithm



Taxi Example: Optimization Algorithm



Taxi Example: Optimization Algorithm



Optimal Number of Taxis is 30

Bayesian Optimization: Next Point to Evaluate

Bayesian optimal design: find a point that maximizes a utility function

$$\theta^{+} = \arg\max_{\theta \in D} \; E[U(\theta, L, \gamma)]$$

Where L is predictive output (loss) under the GP model at point θ

$$E[U(\theta, L, \gamma))] = \int_{y} U(\theta, L, \gamma) p(L \mid \theta, \gamma) dy$$

For a fully Bayesian approach

$$E[U(\theta, L, \gamma))] = \int_{\gamma} \int_{L} U(\theta, L, \gamma) p(L \mid \theta, \gamma) p(\gamma) dL d\gamma$$

Bayesian Optimization: Next Point to Evaluate

$$E[U(\theta, L, \gamma))] = \int_L U(\theta, L, \gamma) p(L \mid \theta, \gamma) dL$$

We can account for the size of improvement (Expected Improvement)

$$U(\theta) = \max(0, L^* - L(\theta))$$

Another recent alternative is upper confidence bound

$$a_{\mathrm{UCB}}(\theta;\ \beta) = m(\theta) - \beta\sigma(\theta)$$

Explicit exploitation-exploration trade-off, parametrized by β. Extra hyper-parameter!

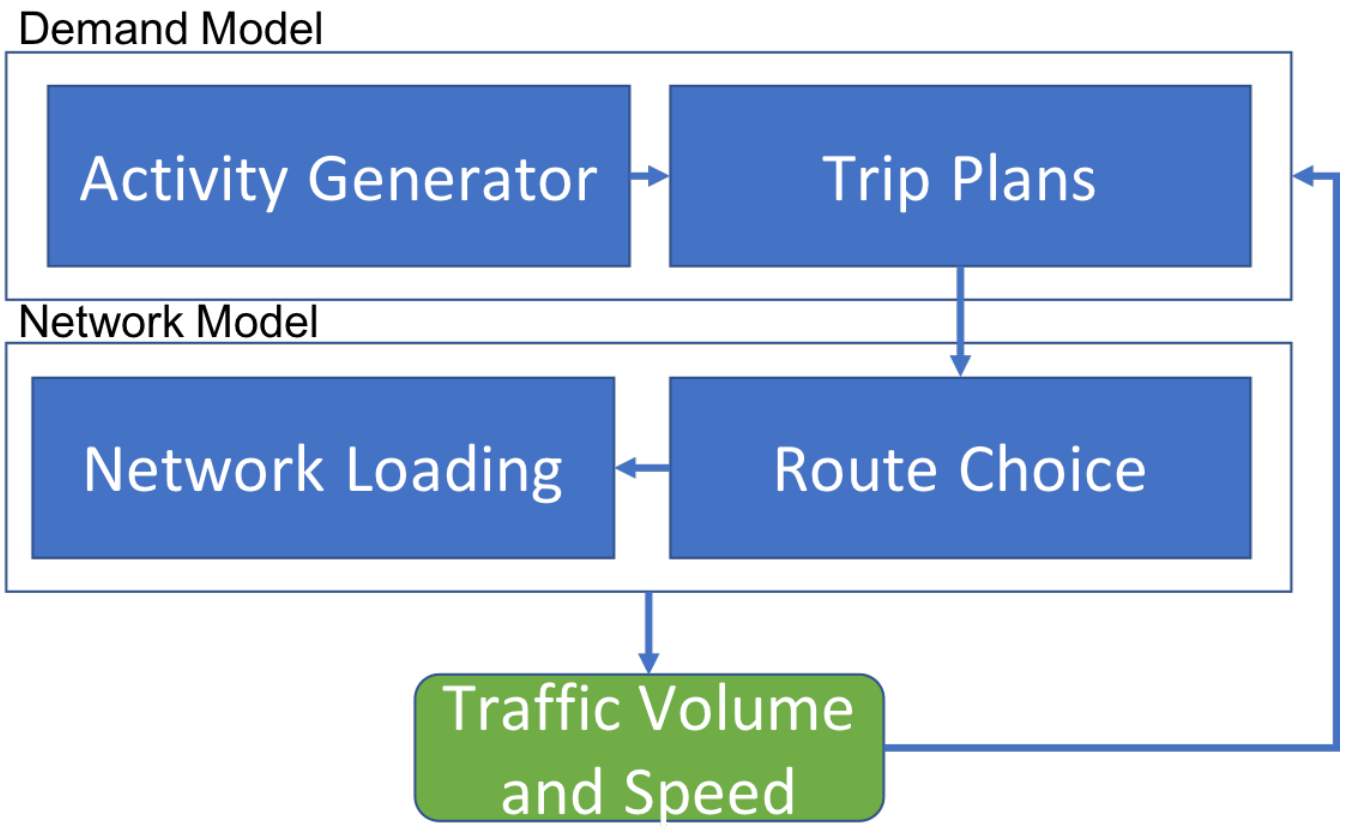
Complex Transportation Models

The models that need calibration cover cities, not blocks

Detroit has 28,000 road segments with 15 million trips

- Each run takes hours to days on a fast desktop
- θ is often very high dimensional and sensitive

Transportation ABM Example: POLARIS



- Activity-based demand generation
- Static variables θ include probability of activities, distribution settings for assigning attributes, etc.

High Dimensional θ

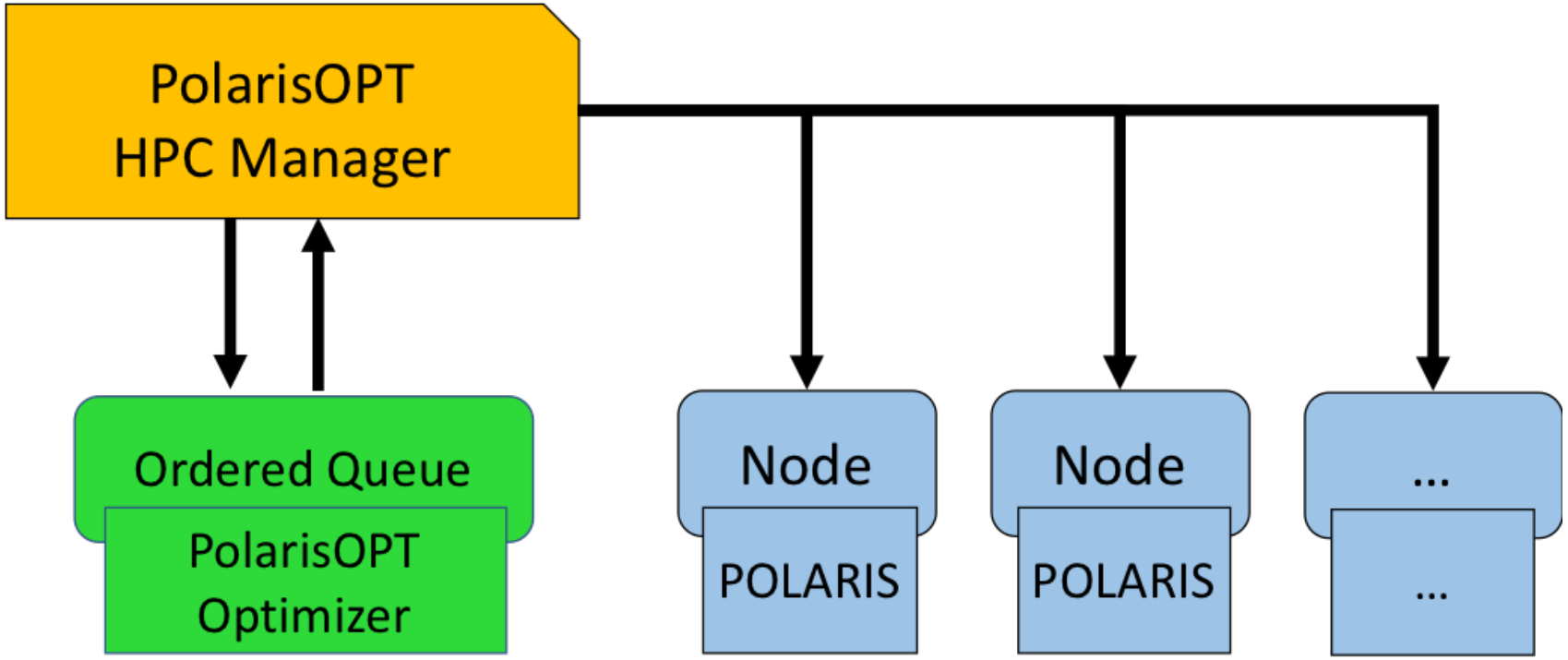
- Exploit model structure
- Better design (e.g. global + local acquisition function)
- Make simulator run faster (e.g. reduced order model)
- Parallelize
- Reduce dimensionality (Higdon, 2008)
- More flexible mean and covariance (Gramacy 2009, Gramacy 2011)

Curse of dimensionality in 20 dimensions with 1 second per run

- grid with 10 points per dimension $10^{20} = 3.171$ trillion years
- evaluate at 2^{20} corners $= 12.14$ days

Even checking for optimality is troublesome, there e^{20} directions whose pairwise angle is > 60 degrees $= 5615$ days

Computational Framework for HPC



HPC Manager:

- Submits simulation jobs to compute nodes
- Tracks the execution status. Resubmits failed jobs and postprocesses outputs of successful jobs

`Sfit/T` based scripts on Linux and vanilla `Python` on Window

Batch Optimization

- Acquisition function gives 1 point to evaluate
- What if we have a parallel machine and can run multiple jobs at once?
- Instead of using k best points we use approach that prefers to explore
- Expected information gain is used instead

Dimensioanlity Reduction

Basic idea

$$\theta \approx g(\psi(\theta)), \quad \psi : R^n \to R^m, \quad m < n$$

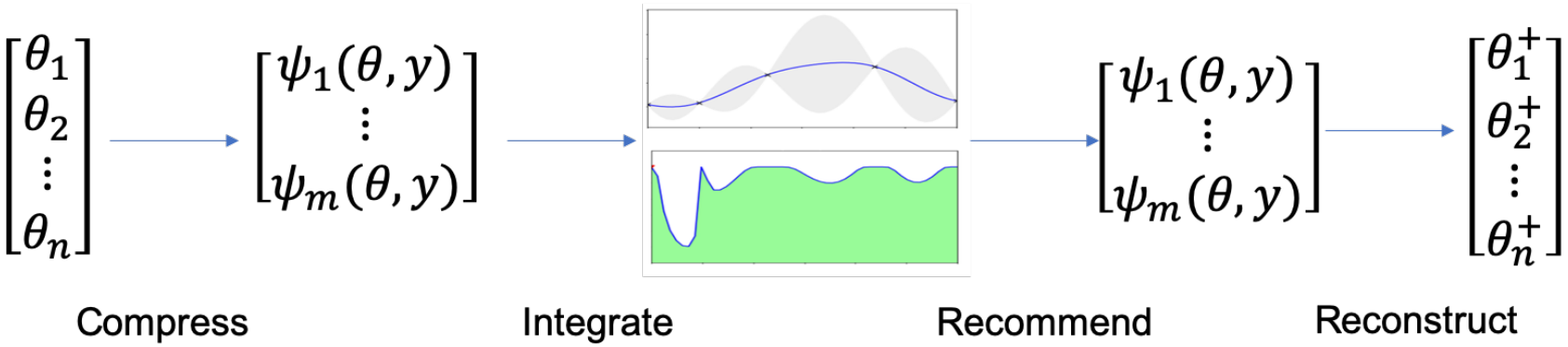
Given $\Theta = (\theta_1, \ldots, \theta_K)$, $Y = (y_1, \ldots, y_K)$

- PCA: $\psi(\theta) = \Psi^T \theta$, $\hat{\Psi} = \arg\min_{\Psi \in S_m} \|\Theta - \Psi\Psi^T\Theta\|_F^2$, $S_m = \{\Psi \in R^{n \times m} \mid \Psi^T \Psi = I\}$
- PLS: $\hat{\Psi} = \arg\max_{\Psi \in S_m} \mathrm{Cov}(\Psi^T \Theta, y)$
- Active Subspace: $\psi(\theta) = \Psi^T \theta$, Ψ are loadings of the gradient of the simulator $\phi(\theta)$
- Our approach:

$$\hat{W} = \arg\min_W \|\theta - g(\psi(\theta))\|_2 + \|Y - q(\psi(\theta))\|_2$$

g, ψ, q are neural networks parametrized by W, we estimate those jointly

Bayesion Optimizaiton in Projected Space



For our framework, 2 objectives exist:

- Capture lower-dimensional structure ($m < n$)
- Reconstruct any recommendation to original dimension

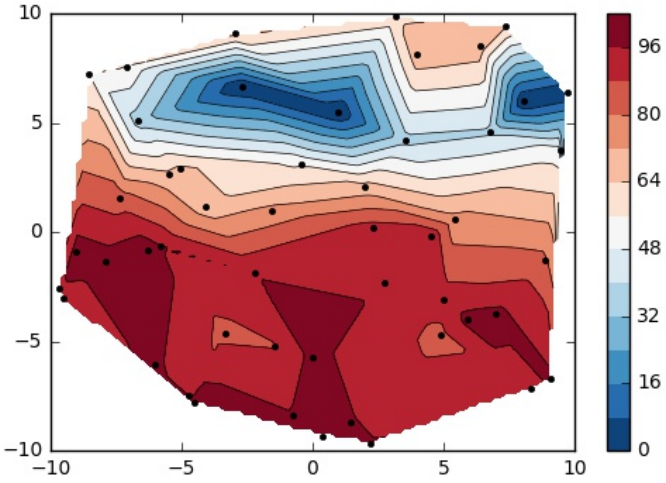
$$\begin{aligned}\mu(x \mid D) &= \mu_0(x) + \mathbf{m}^T\psi(x)\\ \sigma^2(x \mid D) &= \phi(x)^T K^{-1}\psi(x) + 1/\beta,\end{aligned}$$

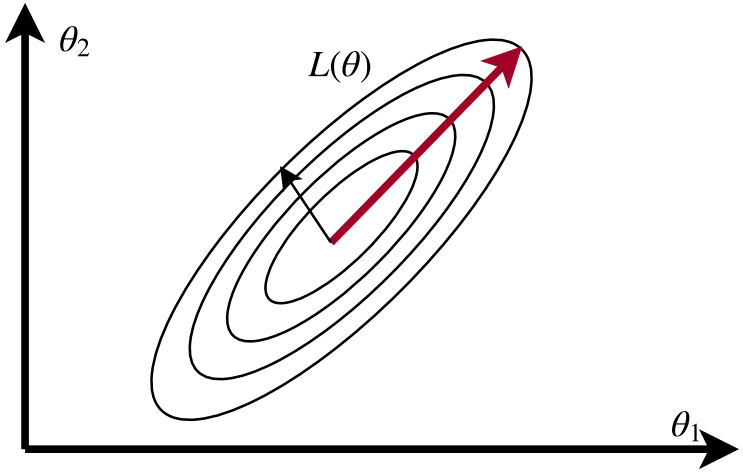
where

$$\begin{aligned}\mathbf{m} &= \beta K^{-1}\Phi^T\tilde{y}\\ K &= \beta\Psi^T\Psi + I\alpha.\end{aligned}$$

Dimensionality Reduction: Active Subspaces

Main idea: Apply PCA to function gradients [Russi 2010]





- Generalization of approach that picks most sensitive inputs
- Replace individual inputs by linear combinations

Dimensionality Reduction: Active Subspaces

$$C = \int (\nabla_\theta L)(\nabla_\theta L)^T d\theta = W \Lambda W^T$$

C is a sum of semi-positive definite rank-one matrices. Partition the eigenvector matrix $W = [W_1, W_2]$, eigenvectors W_1 correspond to m largest eigenvalues

Project and separate the subspaces

$$\theta = \Psi \Psi^T \theta = \Psi_1 \Psi_1^T \theta + \Psi_2 \Psi_2^T \theta$$

$\theta_a = \Psi_1^T \theta$ is active subspace and Ψ_1 is the reconstruction operator

$$\int (\nabla_{\theta_a} L)(\nabla_{\theta_a} L)^T d\theta = \lambda_1 + \ldots + \lambda_m$$

Dimensionality Reduction: Active Subspaces

- Use Monte Carlo to approximate variance of the gradient
- Uniformly sample $\theta_1, \ldots, \theta_N$ and compute $\nabla_\theta L_j = \nabla_\theta L(\theta_j)$

$$C = \int (\nabla_\theta L)(\nabla_\theta L)^T d\theta \approx \frac{1}{N} \sum_{j=1}^{N} (\nabla_\theta L_j)(\nabla_\theta L_j)^T = \hat{W} \hat{\Lambda} \hat{W}^T \hat{C}$$

- Can use bootstrap to estimate variance of $\hat{C}$
- Due to Gittens and Trop (2011)

$$\text{If } N = \Omega\left(\frac{M\lambda_1}{\lambda_k^2 e^2} \log n\right), \text{ then } |\lambda_k - \hat{\lambda}_k| \leq e\lambda_k,$$

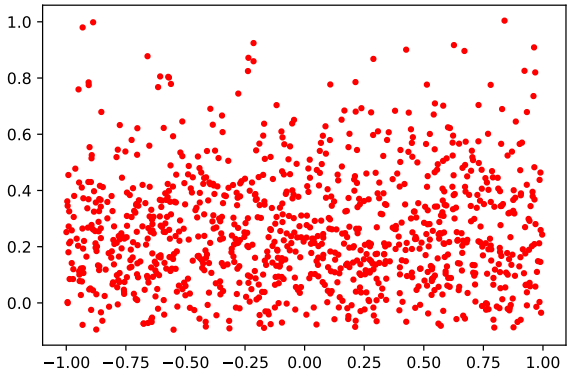
$$\mathrm{dist}(W_1, \hat{W}_1) \leq \frac{4\lambda_1 e}{\lambda_m - \lambda_{m+1}}$$

Dimensionality Reduction: Autoencoder

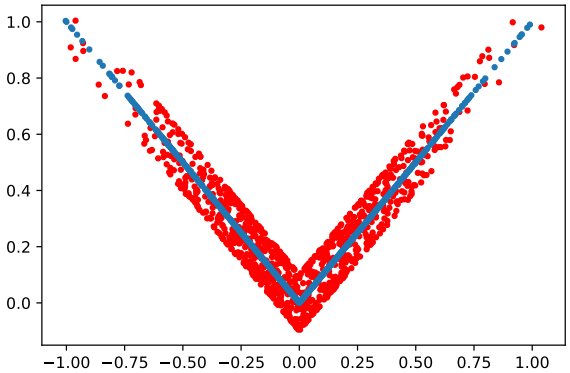
Consider a simple example

$$f(x) = |u^T x| + 0.1 * \sin(100x_2)$$

u is samples uniformly in [-1,1], $x \in R^{100}$, $f : R^{100} \rightarrow R$



x_0 vs $f(x)$

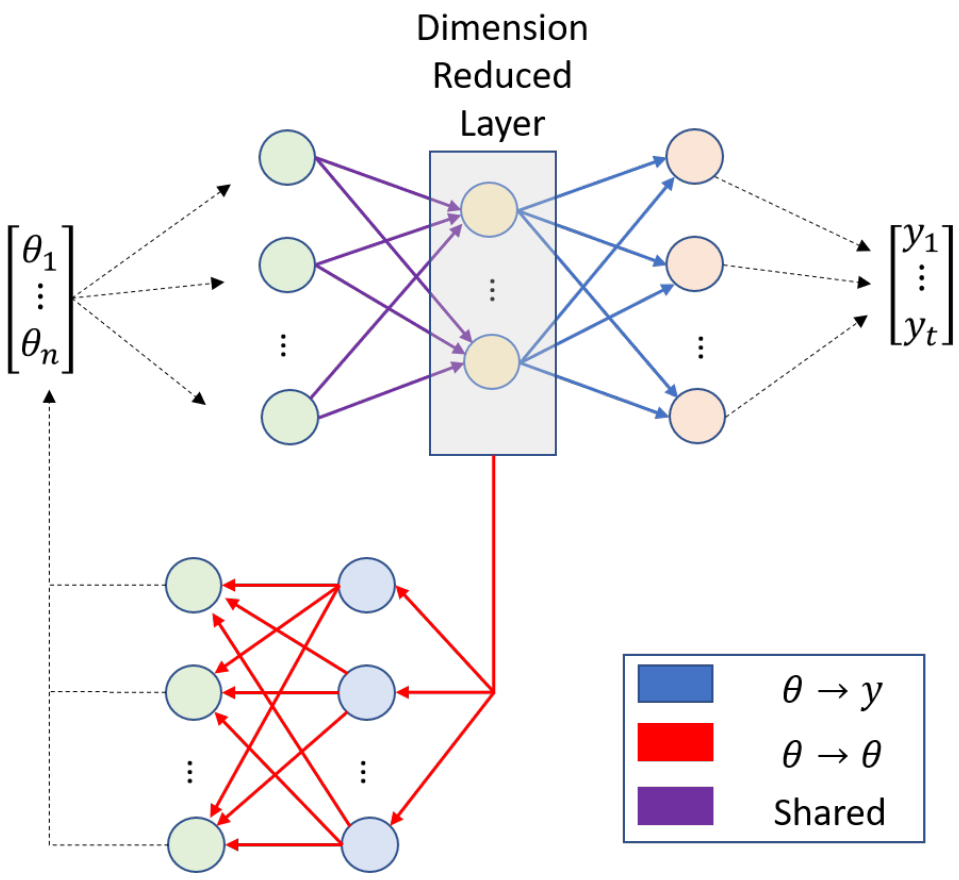


F_{DL} reconstruction

$$F_{DL}(x) = ReLU(w^T x) + ReLu(-w^T x),\ ReLu(z) = \max(z, 0)$$

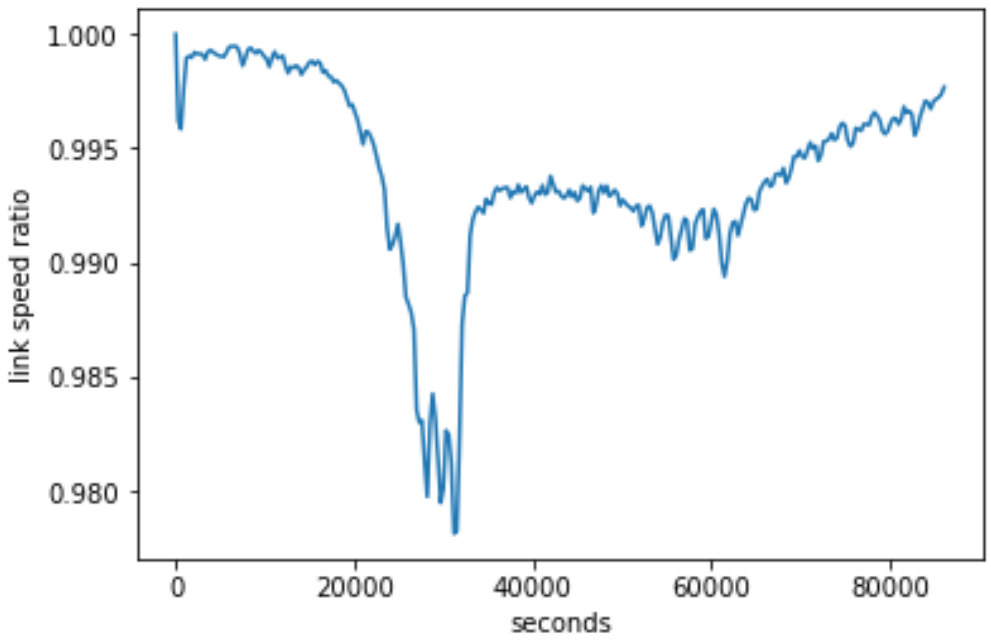
Dimensionality Reduction: Nonlinear

- More flexible projections (non-linearities)
  - Ability to capture local patterns
  - Ignoring smaller variational directions no longer an issue
- No need for derivatives



Time Dependency

Traffic trends are time-dependent

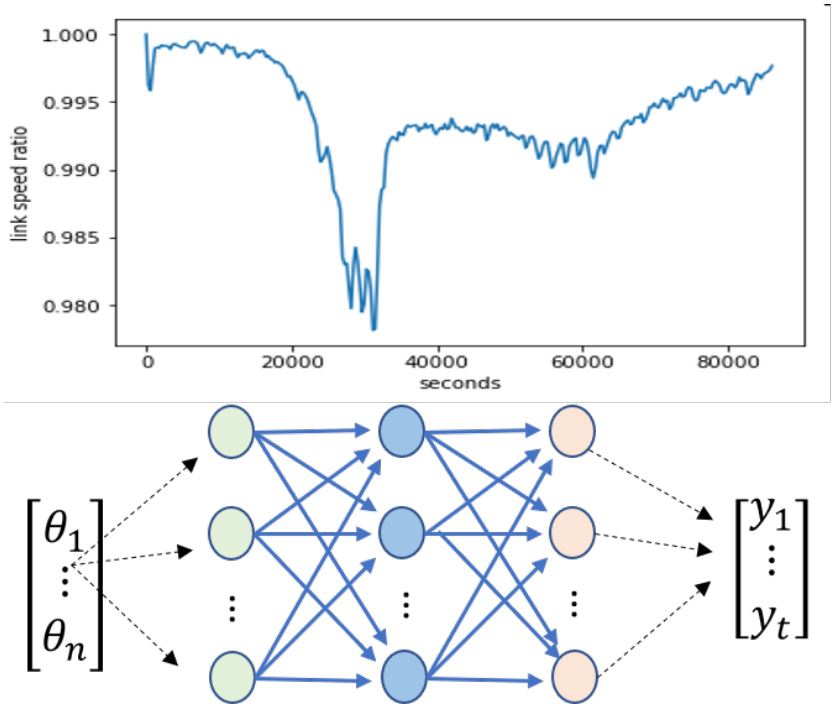


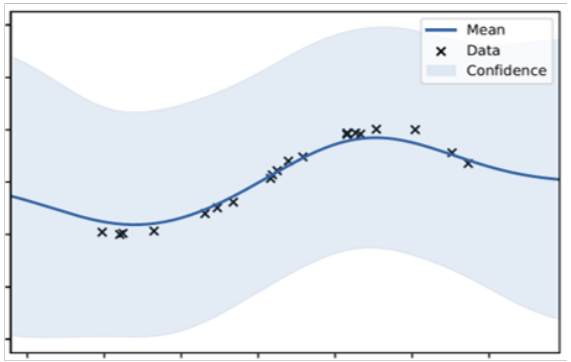
Thus far, we **implicitly** account for it

- Our objective function currently averages over space and time
- Deep Learner learns and captures in smaller dimension representation

Mean Improvement

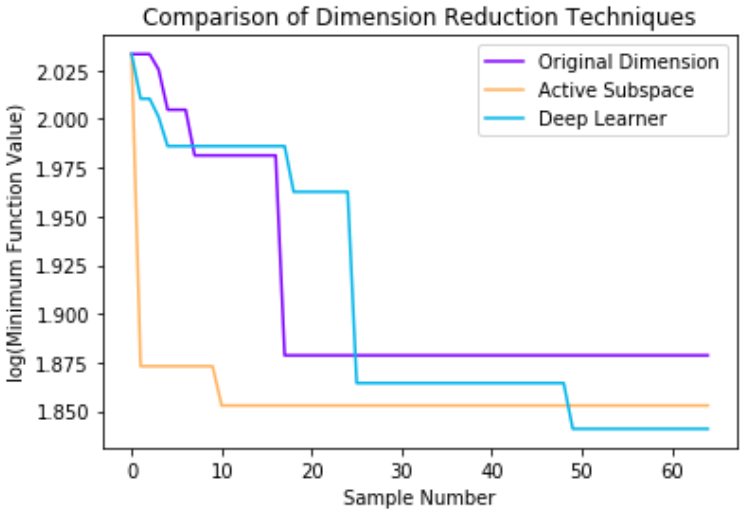
However, a Deep Learner mean function is an opportunity to **explicitly** capture this relationship



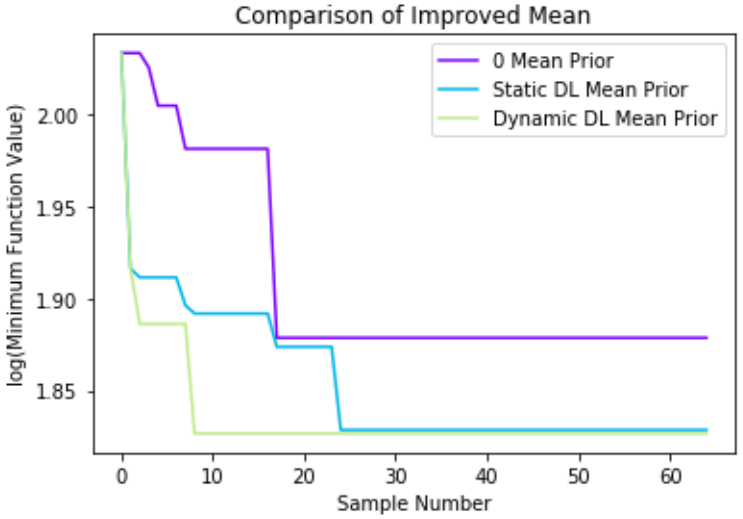


- GP mean function influences predicted posterior **and** acquisition function
- GP standard assumption was to set mean to zero

Results



(a) Dimensional Reduction



(b) Mean Function

	Original Dimension (n=9)	Active Subspaces (n=2)	Deep Neural Network (n=4)
$\mathcal{GP}(m(\theta) = 0, \mathrm{K})$	75.61	72.26	69.71
$\mathcal{GP}(m(\theta) = DL_{Static}, \mathrm{K})$	67.39	65.88	56.14
$\mathcal{GP}(m(\theta) = DL_{Dynamic}, \mathrm{K})$	67.08	68.9	68.04

References

- Gramacy, R. B., & Lee, H. K. (2012). Cases for the nugget in modeling computer experiments. Statistics and Computing, 22(3), 713-722.
- Srinivas, N., Krause, A., Kakade, S. M., & Seeger, M. (2009). Gaussian process optimization in the bandit setting: No regret and experimental design. arXiv preprint arXiv:0912.3995.
- Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical bayesian optimization of machine learning algorithms. In Advances in neural information processing systems (pp. 2951-2959).
- Higdon, D., Gattiker, J., Williams, B., & Rightley, M. (2008). Computer model calibration using high-dimensional output. Journal of the American Statistical Association, 103(482), 570-583.
- Russi, T. M. (2010). Uncertainty quantification with experimental data and complex system models (Doctoral dissertation, UC Berkeley)
- Gittens, A., & Tropp, J. A. (2011). Tail bounds for all eigenvalues of a sum of random matrices. arXiv preprint arXiv:1104.4513.

Discussion

- Gaussian process is a practical surrogate for transportation agent-based models
- High dimensionality can be treated using Active Subspace (requires derivatives) or DL
- Deep learning surrogate is less understood but leads to better results
- Next steps:
  - DL dimensionality reduction
  - combine GP with DL for surrogate model
  - opening up the black box (utilize network structure, do not approximate demand simulator)